

COMPUTER ORGANIZATION & ARCHITECTURE (COA) SOLVED BANK

SISTec Academic Mid-Semester Pattern — Semester IV

Unit 1 (Basic Structure of Computer) & Unit 2 (Computer Arithmetic & Control Memory)

Subject: Computer Organization & Architecture (CSIT-404)

Total Solved Marks: 28 Marks

Layout Structure: Part A (MCQ), Part B (Short), Part C (Long Choice)

Branding Highlight: EMo Learners Exclusive
Solved Paper

PART A: OBJECTIVE MULTIPLE CHOICE QUESTIONS (6 MARKS)

Instructions: Answer all twelve questions. Each question carries exactly 0.5 marks.

Q.1 What is meant by the Von-Neumann bottleneck?

- A. CPU processing cycles are too slow
- B. Shared bus restricts simultaneous data and instruction fetch
- C. Registers are too small to store data
- D. Multiplexer selection delays the ALU

Answer: B. Shared bus restricts simultaneous data and instruction fetch

Explanation: Since instructions and data share the same physical bus in the Von-Neumann architecture, the CPU cannot read an instruction and read/write data at the same instant, throttling throughput.

Q.2 In register transfer language (RTL), what does the expression ' $P : R2 \leftarrow R1$ ' represent?

- A. R1 loads R2 at all clock pulses
- B. R1 and R2 swap values on condition P
- C. If Boolean condition $P = 1$, R2 loads contents of R1
- D. R2 is cleared if $P = 0$

Answer: C. If Boolean condition $P = 1$, R2 loads contents of R1

Explanation: P is a control function (Boolean condition). When P evaluates to 1, the target microoperation (transferring R1 to R2) is executed at the next clock edge.

Q.3 Which addressing mode does not require a memory reference to fetch the operand?

- A. Direct Addressing
- B. Indirect Addressing
- C. Immediate Addressing
- D. Relative Addressing

Answer: C. Immediate Addressing

Explanation: In immediate addressing, the operand is explicitly defined inside the instruction word itself, requiring zero memory access cycles to fetch data.

Q.4 Which CPU register holds the address of the next instruction word to be fetched from memory?

- A. Instruction Register (IR)
- B. Program Counter (PC)
- C. Accumulator (AC)
- D. Memory Address Register (MAR)

Answer: B. Program Counter (PC)

Explanation: The Program Counter (PC) is a dedicated register that points to and tracks the next instruction's address in system memory.

Q.5 How many 8-to-1 multiplexers are required to construct a common bus system for eight 16-bit registers?

- A. 8 multiplexers
- B. 16 multiplexers
- C. 24 multiplexers
- D. 128 multiplexers

Answer: B. 16 multiplexers

Explanation: A common bus system requires exactly one multiplexer for each bit line of the register size. For 16-bit registers, we need 16 multiplexers (each being 8-to-1 since there are 8 registers).

Q.6 What is the range of signed integers that can be stored in an 8-bit register using 2's complement representation?

- A. -127 to +127
- B. -128 to +127
- C. -128 to +128
- D. 0 to 255

Answer: B. -128 to +127

Explanation: Using n bits, 2's complement represents values from $-2^{(n-1)}$ to $+(2^{(n-1)} - 1)$. For 8 bits, this yields -128 to +127, featuring a unique single representation of zero.

Q.7 In Booth's multiplication algorithm, what action is taken when the multiplier bits Q_0 and Q_{-1} are equal to '10'?

- A. Shift only (ARS)
- B. Add multiplicand to accumulator, then shift
- C. Subtract multiplicand from accumulator, then shift
- D. Clear the accumulator

Answer: C. Subtract multiplicand from accumulator, then shift

Explanation: Booth's algorithm rules specify: 00 or 11 = shift only; 01 = add and shift; 10 = subtract ($A = A - M$) and shift.

Q.8 What is the biased exponent offset value used in the standard IEEE 754 Single-Precision floating-point format?

- A. 127
- B. 128
- C. 1023
- D. 255

Answer: A. 127

Explanation: IEEE 754 single-precision format uses 8 bits for the exponent with a bias of 127, allowing representation of exponents from -126 (stored as 1) to +127 (stored as 254).

Q.9 What timing component in a hardwired control unit is used to generate sequential timing state signals T0, T1, etc.?

- A. Instruction Decoder
- B. Step Sequence Counter
- C. Control Address Register
- D. Control Memory ROM

Answer: B. Step Sequence Counter

Explanation: A step counter increments continuously on clock edges, feeding into a decoder that generates sequential timing states (T0, T1, T2) to guide microoperations.

Q.10 Where are the control words (microinstructions) stored in a microprogrammed control unit?

- A. Primary RAM
- B. CPU General Registers
- C. Control Memory (Internal ROM)
- D. Instruction Register

Answer: C. Control Memory (Internal ROM)

Explanation: In a microprogrammed control unit, control signals are pre-written as microcode and stored in a high-speed internal Control Memory ROM.

Q.11 Which microinstruction organization offers higher parallel execution capabilities and faster operating speed?

- A. Vertical Microinstruction
- B. Horizontal Microinstruction
- C. Encoded Microinstruction
- D. Indirect Microinstruction

Answer: B. Horizontal Microinstruction

Explanation: Horizontal microinstructions dedicate a separate bit to each physical control line. This eliminates decoding delays, allowing multiple microoperations to execute in parallel instantly.

Q.12 In Restoring Binary Division, what action is taken when the partial remainder in register A becomes negative (MSB = 1) after subtraction?

- A. Set quotient bit Q0 = 1
- B. Add divisor back to A to restore its state, and set Q0 = 0
- C. Arithmetic right shift A-Q
- D. Terminate the division loop

Answer: B. Add divisor back to A to restore its state, and set Q0 = 0

Explanation: Restoring division subtracts the divisor. If the result is negative, it must 'restore' the original value by adding the divisor back, setting the quotient bit Q0 to 0.

PART B: SHORT ANSWER QUESTIONS (8 MARKS)**Section 1: Unit 1 (Basic Structure of Computer)**

Instructions: Answer any two of the following three questions. Each question carries 2 marks.

Q.1 Compare Direct and Indirect addressing modes with suitable examples.

[□ Marks: 2.0]

Answer:

Direct and Indirect addressing modes are compared below in a structured table:

Parameter	Direct Addressing Mode	Indirect Addressing Mode
Effective Address (EA)	EA = Address Field (A). Directly points to operand location in memory.	EA = M[A]. Points to a pointer register/memory containing actual EA.
Memory References	1 reference to read operand.	2 references (1st to read pointer, 2nd for actual operand).
Execution Speed	Fast due to fewer memory access cycles.	Slower due to overhead of double memory access.
Flexibility	Limited; hard to implement dynamic reference structures.	High; essential for array pointers and list traversals.
Assembly Example	`ADD 250H` → Adds value at address 250H directly to AC.	`ADD [250H]` → Reads 250H for pointer, adds that target to AC.

Q.2 Differentiate between Hardwired and Microprogrammed control units with respect to design complexity and flexibility.

[□ Marks: 2.0]

Answer:

Hardwired and Microprogrammed control units are compared in the table below:

Parameter	Hardwired Control Unit	Microprogrammed Control Unit
Construction	Built using **combinational hardware circuits** (decoders, gates, step-counters).	Built using a systematic **Control Memory ROM** storing microcode words.
Complexity	Extremely high and customized, grows with instruction set size.	Systematic and clean, modular instruction sequencing logic.
Operating Speed	Very fast (gate propagation delays only).	Slower due to memory read cycle latency of Control ROM.

Flexibility	Very rigid; modifying instructions requires physical rewiring.	Highly flexible; can add instructions by updating microcode in ROM.
Cost & Application	High development cost; preferred in RISC cores for raw speed.	Lower design cost; widely used in complex instruction CISC cores.

Q.3 Define the Von-Neumann Bottleneck and its main causes. List two mitigation techniques.

[□ Marks: 2.0]

Answer:

The Von-Neumann Bottleneck is a throughput limitation where CPU processing speeds are severely throttled because the data/instruction throughput of the shared bus is much slower than internal CPU processing speeds.

- Causes: Instructions and data share the exact same physical memory bus. The CPU cannot perform an instruction fetch and a data read/write at the same instant.

• Mitigation Techniques:

1. **Cache Memories:** Integrating high-speed L1/L2/L3 SRAM caches inside the CPU.
2. **Harvard Architecture:** Splitting instruction caches and data caches into physically separate structures.

Section 2: Unit 2 (Computer Arithmetic & Control Memory)

Instructions: Answer any two of the following three questions. Each question carries 2 marks.

Q.4 Define 1's complement and 2's complement representation and list their advantages in signed arithmetic operations.

[□ Marks: 2.0]

Answer:

- 1's Complement: Formed by inverting all bits of the positive integer (0 to 1, 1 to 0).
- 2's Complement: Formed by adding 1 to the Least Significant Bit (LSB) of the 1's complement representation.

• Advantages in Signed Arithmetic:

1. No Subtraction Circuitry Needed: Subtraction is performed simply by adding the 2's complement of the subtrahend ($A - B = A + (-B)$).
2. Unique Representation of Zero: 2's complement has a single representation for zero ('00000000'), avoiding the ambiguous '+0' and '-0' found in 1's complement. This significantly simplifies ALU hardware zero-testing logic.

Q.5 Analyze the difference between fixed-point and floating-point number representation in terms of range and precision.

[□ Marks: 2.0]

Answer:

Fixed-point and Floating-point number representations are compared below:

Parameter	Fixed-Point Representation	Floating-Point Representation
Radix Point Position	Fixed at a constant position (e.g. at the end for integers).	Floats dynamically; represented in scientific notation ($M \times R^E$).
Range of Values	Narrow and limited range for a given register bit width.	Extremely large range (handles tiny fractions and massive values).
Precision	Uniform and constant spacing between representable values.	Variable precision (high near zero, decreases for larger numbers).
Hardware Cost	Low; simple adder and shift circuits.	High; requires dedicated **Floating Point Unit (FPU)** logic.
Arithmetic Speed	Fast and direct execution cycles.	Slower due to alignment, normalization, and rounding steps.

Q.6 Explain the concept and advantages of a biased exponent in IEEE 754 floating-point representation.

[□ Marks: 2.0]

Answer:

The standard IEEE 754 format uses a biased exponent (bias of 127 for 8-bit single-precision exponents) to allow representation of both positive and negative powers of 2 without requiring a sign bit inside the exponent field.

- Simplification of Comparisons: Representing exponents as unsigned positive numbers (ranging from 1 to 254, corresponding to actual exponents -126 to +127) allows high-speed hardware comparators to compare floating-point magnitudes directly using standard integer comparison logic, without needing complex signed math operators.

PART C: LONG ANSWER & TRACING QUESTIONS (14 MARKS)

Set 1: Unit 1 (Basic Structure of Computer)

Instructions: Answer any one of the following two questions (with internal choice). The question carries 7 marks.

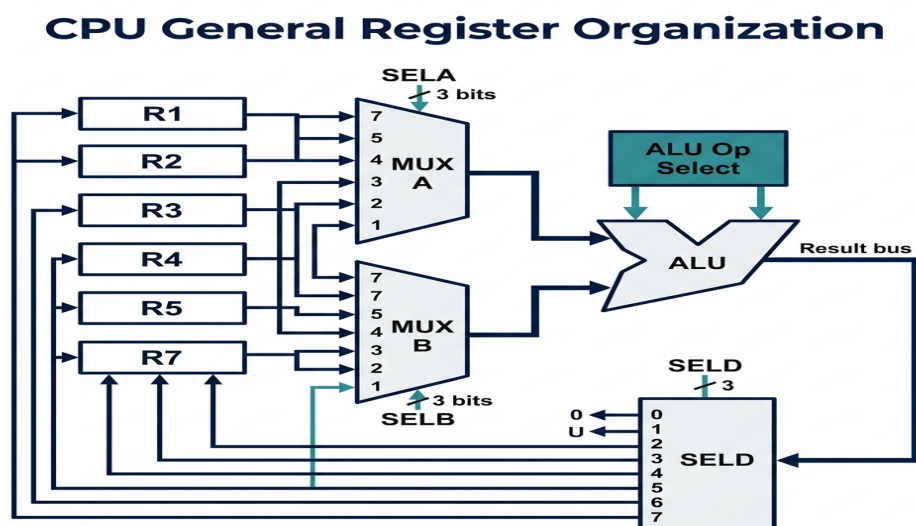
Q.7 [a] Explain the general register organization of the CPU with a neat block diagram and detail the step-by-step role of ALU and internal buses in executing $R1 \leftarrow R2 + R3$.

[□ Marks: 7.0]

Answer:

In a General Register Organization, multiple registers are connected through a common internal bus system feeding into a central ALU.

1. Bus-based CPU Register Organization Schematic:



2. Step-by-Step Execution of $R1 \leftarrow R2 + R3$:

To execute the addition microoperation in a single clock cycle, the control unit coordinates the internal buses as follows:

- **Select Source Operands (T1):** The control unit configures the multiplexer select lines ****SELA**** to place the contents of R2 onto Bus A, and ****SELB**** to place the contents of R3 onto Bus B.
- **ALU Operation Selection (T2):** The control unit configures the ALU operation selector ****OPR**** to perform addition.
- **Write Back Result (T3):** The addition sum is placed on the Result Bus C. The control unit configures the destination decoder ****SELD**** to enable the ****Load (LD)**** line of register R1. On the next active clock edge, R1 locks in the sum.

----- OR -----

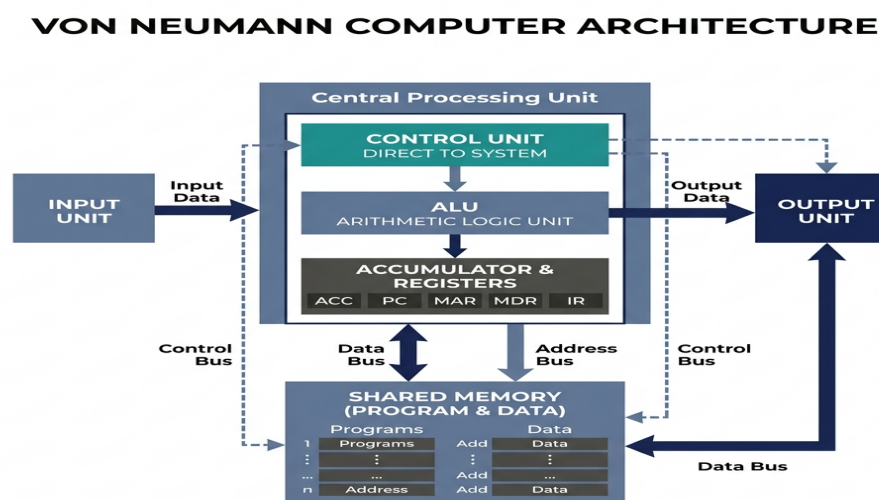
Q.7 [b] Explain the structure of a desktop computer with a neat block diagram and describe the function of each major unit (Input, Output, Memory, CPU Control Unit, and ALU).

[□ Marks: 7.0]

Answer:

A desktop computer's logical architecture is built on the classic Von Neumann Architecture, containing the CPU, Memory, and Input/Output units.

1. Von Neumann Architecture Hardware Block Diagram:



2. Key Functional Units and Their Roles:

- **Input Unit:** Translates human and external physical signals (from keyboard, mouse, scanners) into binary codes that can be loaded into system memory and CPU.
- **Output Unit:** Converts binary values processed by the computer back into physical human-readable forms (monitors, printers, speakers).
- **Memory Unit (Main Memory / RAM):** Stores active programs, variable data, and instructions currently being processed by the CPU. The address bus points to words, and the data bus fetches/writes them.
- **Arithmetic & Logic Unit (ALU):** Performs the core mathematical operations (addition, subtraction, multiplication) and relational logical comparisons (AND, OR, NOT, comparisons).
- **Control Unit (CU):** Acts as the nervous system of the CPU. It continuously fetches instructions from memory, decodes their operation fields, and generates timing strobe signals to manage registers, ALU, memory, and bus flows.

Set 2: Unit 2 (Computer Arithmetic & Control Memory)

Instructions: Answer any one of the following two questions (with internal choice). The question carries 7 marks.

Q.8 [a] Explain Booth's multiplication algorithm. Trace the step-by-step signed multiplication process to multiply (+13) and (-9) in 6-bit binary, showing the Accumulator, Multiplier, and Q_{-1} registers.

[□ Marks: 7.0]

Answer:

Let us multiply $(+13) \times (-9)$ using Booth's Algorithm in 6-bit signed 2's complement representation:

- Multiplicand (M) = $+13 = 001101_2$
 - Negative Multiplicand (-M) = $-13 = 110011_2$ (2's complement of 001101)
 - Multiplier (Q) = $-9 = 110111_2$ (2's complement of 001001)
- **Registers:** Accumulator (A) = 000000 (6 bits), $Q_{-1} = 0$ (1 bit), SC = 6.

Cycle-by-Cycle Tracing Table:

Cycle / Action Step	Accumulator (A)	Multiplier (Q)	Q_{-1}	SC
Initial State	000000	110111	0	6
Cycle 1: $Q_0Q_{-1} = 10 \rightarrow A = A - M$ ARS only	110011 111001	110111 111011	0 1	6 5
Cycle 2: $Q_0Q_{-1} = 11 \rightarrow$ ARS only	111100	111101	1	4
Cycle 3: $Q_0Q_{-1} = 11 \rightarrow$ ARS only	111110	011110	1	3
Cycle 4: $Q_0Q_{-1} = 01 \rightarrow A = A + M$ ARS only	001011 000101	011110 101111	1 0	3 2
Cycle 5: $Q_0Q_{-1} = 10 \rightarrow A = A - M$ ARS only	111000 111100	101111 010111	0 1	2 1
Cycle 6: $Q_0Q_{-1} = 11 \rightarrow$ ARS only	111110	001011	1	0

Result Verification:

The final combined register contents are A Q = 111110001011₂ (12 bits signed).

Since the MSB is 1, the result is negative. Let's find its magnitude via 2's complement:

1. Invert all bits $\rightarrow 000001110100$
2. Add 1 $\rightarrow 000001110101_2$
3. Convert to decimal $\rightarrow 2^6 + 2^5 + 2^4 + 2^2 + 2^0 = 64 + 32 + 16 + 4 + 1 = 117$.

Therefore, the value is -117. Multiplying $(+13) \times (-9)$ yields -117. The trace is correct!

----- OR -----

Q.8 [b] Perform binary division of 11 divided by 4 using the Restoring division method with a step-by-step hardware trace.

[□ Marks: 7.0]

Answer:

In Restoring Division, the divisor M is subtracted from A in each cycle. If the MSB of A is 1 (negative result), the original state is 'restored' by adding M back to A, and the quotient bit Q_0 is set to 0. Otherwise, the remainder remains active and Q_0 is set to 1.

Step-by-Step Hardware Tracing Table (11 ÷ 4):

Cycle / Action Step	Accumulator (A)	Multiplier (Q)	SC
Initial State (Dividend Q=1011, Divisor M=0100, -M=1100)	0000	1011	4
Cycle 1: 1. Shift Left AQ 2. Subtract M ($A \leftarrow A + (-M)$) 3. Remainder is negative (MSB=1) → Set $Q_0 = 0$ 4. Restore A ($A \leftarrow A + M$)	0001 1101 1101 0001	011_ 011_ 0110 0110	4 - - 3
Cycle 2: 1. Shift Left AQ 2. Subtract M ($A \leftarrow A + (-M)$) 3. Remainder is negative (MSB=1) → Set $Q_0 = 0$ 4. Restore A ($A \leftarrow A + M$)	0010 1110 1110 0010	110_ 110_ 1100 1100	3 - - 2
Cycle 3: 1. Shift Left AQ 2. Subtract M ($A \leftarrow A + (-M)$) 3. Remainder is positive (MSB=0) → Set $Q_0 = 1$ 4. No Restore	0101 0001 0001 0001	100_ 100_ 1001 1001	2 - - 1
Cycle 4: 1. Shift Left AQ 2. Subtract M ($A \leftarrow A + (-M)$) 3. Remainder is negative (MSB=1) → Set $Q_0 = 0$ 4. Restore A ($A \leftarrow A + M$)	0011 1111 1111 0011	001_ 001_ 0010 0010	1 - - 0

Final Verification:

- Quotient (Q): `0010` → 2 in decimal
- Remainder (A): `0011` → 3 in decimal

Checking our arithmetic: Dividend = (Divisor × Quotient) + Remainder → 11 = (4 × 2) + 3. The tracing process is perfectly correct!